

# *Aula 07: Subprogramação II*

## *Recursividade*

*Maurício da Silva Pires*  
*mspires@id.uff.br*

*TCC00326 - Programação de computadores*  
*Niterói, Rio de Janeiro*

## Revisitando as aulas anteriores

---

- ⌋ Nas aulas anteriores, trabalhamos com o conceito de subprogramação
  - \* Construímos algumas funções em Python
  - \* Aprendemos a manipular módulos
  - \* Vimos que funções podem chamar outras funções
- ⌋ Os algoritmos que desenvolvemos até o momento podem ser definidos como **iterativos**, ou seja, que fazem uso de repetições
- ⌋ Nesta aula, vamos aprender uma nova estratégia de definir funções: a **recursividade**

- ☾ Para estudantes iniciantes, o conceito de programação recursiva costuma ser difícil
  - \* O pensamento recursivo é complexo porque se assemelha a um raciocínio circular
  - \* Além disso, não é um processo intuitivo; quando damos instruções a outras pessoas, raramente as direcionamos recursivamente
- ☾ Para aqueles que são novos na programação de computadores, aqui está uma definição simples de recursão

**A recursão ocorre quando uma função chama a si mesma direta ou indiretamente**

## Relação de recorrência

---

- ﴿ **Relação de recorrência** é uma formulação matemática que permite descrever elementos de uma sequência infinita em função de elementos que apareceram antes no desenvolvimento desta sequência
- ﴿ Com o objetivo de não criar uma definição circular, uma relação de recorrência precisa ter duas características
  - \* É necessário que alguns elementos sejam primariamente elementos da sequência
  - \* Toda vez que a sequência referencia a si mesma, ela deve sempre se aproximar de um caso base
  - \* Estas exigências fazem com que nosso raciocínio parta de um cenário amplo em direção a fragmentos, que é uma estratégia na computação chamada **divisão-e-conquista**

## Relação de recorrência

---

- ☾ Formalmente falando, uma **relação de recorrência** é composta de dois ingredientes essenciais
  - \* **Caso(s) base(s)**: valores inerentemente pertencentes à sequência
  - \* **Equação(ões) de recorrência**: as formulações que fazem referência a elementos previamente existentes na sequência
- ☾ Recursividade cria dois sentidos de dependência
  - \* Partindo de um item em direção à base: faz-se o movimento de desenrolar a recorrência
  - \* Da base de volta para a primeira chamada: faz-se o movimento do cálculo

- ⌋ Seja  $n$  um número natural. Define-se **fatorial de um número  $n$** , denotado por  $n!$ , como o produto de todos os  $n$  primeiros números naturais

$$1! = 1$$

$$2! = 2 \times 1 = 2$$

$$3! = 3 \times 2 \times 1 = 6$$

$$4! = 4 \times 3 \times 2 \times 1 = 24$$

- ⌋ Por conveniência na Análise Combinatória, admite-se que  $n$  seja inteiro não-negativo, de sorte que  $0! = 1$

## Uma formulação básica

---

﴿ Observe seguinte

$$1! = 1$$

$$2! = 2 \times 1 = 2$$

$$3! = 3 \times 2 \times 1 = 6$$

$$4! = 4 \times 3 \times 2 \times 1 = 24$$

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

﴿ Em outras palavras, com exceção do número 1, o fatorial de um número  $n$  qualquer é igual ao produto de  $n$  pelo fatorial de  $n - 1$ , ou seja, de  $n$  pelo fatorial do antecessor de  $n$

## Uma formulação básica

---

- ⌋ Deste modo, podemos expressar o fatorial a partir da seguinte formulação

$$\text{fatorial}(1) = 1$$

$$\text{fatorial}(n) = n \times \text{fatorial}(n - 1), \quad n \geq 2$$

- ⌋ Assim, na formulação recursiva do fatorial, temos que há apenas um caso base (1!) e uma única equação de recorrência
- \* Esta é o que chamamos **relação de recorrência de primeira ordem**
  - \* A **ordem** da relação de recorrência é determinada pelo número de casos base

## Uma formulação básica

---

- Traduzindo a notação matemática para linguagem de programação, temos o seguinte

```
def fatorial(n: int) -> int :  
    if n < 0 : return -1 # sinal de erro  
    if n == 0 : return 1 # conveniência  
    return n * fatorial(n-1)
```

- Escolhemos a base da recursão como 0 por conveniência, caso queiramos usar esse exemplo como base de outras funções combinatórias

## Sequência de Fibonacci

---

- ⌈ A Sequência de Fibonacci é definida matematicamente como

$$\text{fib}(1) = 1$$

$$\text{fib}(2) = 1$$

$$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2), \quad n \geq 3$$

- ⌈ A Sequência de Fibonacci é uma relação de recorrência de segunda ordem (temos duas bases) e o caso recursivo referencia 2 elementos
  - \* A partir do terceiro elemento da sequência, o elemento atual é igual à soma dos dois anteriores

## Sequência de Fibonacci

---

- Traduzindo a notação matemática para linguagem de programação, temos o seguinte

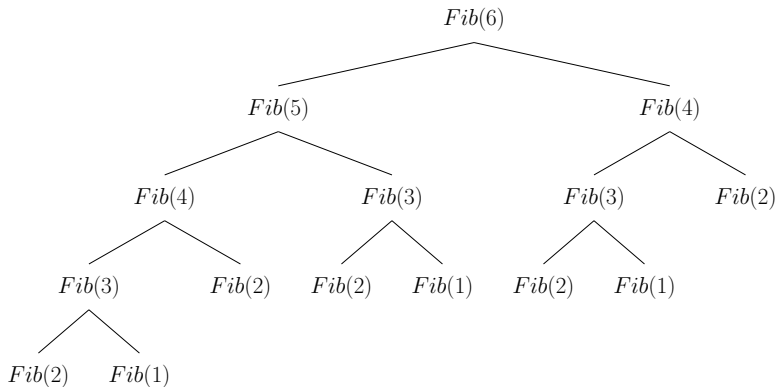
```
def fib(n: int) -> int :  
    if n <= 0 : return -1 # sinal de erro  
    if n == 1 or n == 2 : return 1  
    return fib(n-1) + fib(n-2)
```

- Observe que, nessa formulação, calcular  $\text{fib}(n-1)$  e  $\text{fib}(n-2)$  fará com que recalculemos alguns elementos duas ou mais vezes
  - \* Recursividade não é sinônimo de eficiência
  - \* Fibonacci é um caso em que o cálculo recursivo é menos eficiente que o iterativo

## Sequência de Fibonacci

---

- Observe a árvore de chamadas abaixo para o cálculo do sexto número de Fibonacci



## Outras seqüências recursivas

---

- ⌋ Progressões aritméticas (PA) e geométricas (PG) são outros clássicos exemplos de seqüências recursivas
  - \* Ambas possuem um termo inicial, que é o caso base
  - \* O caso recursivo (recorrente) da PA é somar a razão ao termo anterior
  - \* O caso recursivo (recorrente) da PG é multiplicar a razão pelo termo anterior
- ⌋ **Exercício:** Considere uma progressão de primeiro termo  $a$  e razão  $r$ . Formalize, via relação de recorrência, os casos em que essa progressão seja aritmética e que seja geométrica.

## Outras sequências recursivas

---

﴿ Pensando de uma forma matemática, podemos escrever

$$\begin{cases} a_1 = a \\ a_n = a_{n-1} + r, \quad n \geq 2 \end{cases}$$

$$\begin{cases} a_1 = a \\ a_n = a_{n-1} \times r, \quad n \geq 2 \end{cases}$$

﴿ Pensando de uma forma computacional, podemos escrever

$$ari(n) = \begin{cases} a & \text{se } n = 1 \\ ari(n-1) + r & \text{se } n \geq 2 \end{cases}$$

$$geo(n) = \begin{cases} a & \text{se } n = 1 \\ geo(n-1) \times r & \text{se } n \geq 2 \end{cases}$$

# Dicas para pensar recursivamente

---

## ⌋ Defina o **Caso Base** primeiro

- \* Esta é a condição de parada
- \* É a versão mais simples do problema, onde a resposta é direta e não exige nova recursão

## ⌋ Garanta a aproximação do **Caso Base**

- \* A cada chamada recursiva, os parâmetros de entrada devem mudar e se mover em direção ao caso base

## ⌋ Use o pensamento de cima para baixo

- \* Finja que a função já funciona para subproblemas menores
- \* *Se eu já tivesse o resultado da etapa anterior, o que eu precisaria fazer com ele agora para resolver o problema atual?*

## Revisando o conteúdo

---

- ⌋ A recursão ocorre quando uma função chama a si mesma direta ou indiretamente
- ⌋ Toda função recursiva é composta de dois elementos básicos
  - \* **Caso base:** Cenário(s) em que a resposta da função é imediata
  - \* **Caso recursivo:** Cenário(s) generalizado(s) que referencia elementos anteriores
- ⌋ Dicas para elaborar uma função recursiva
  - \* Definir um Caso Base primeiro
  - \* Garanta a aproximação do Caso Base
  - \* Use o pensamento de cima pra baixo

